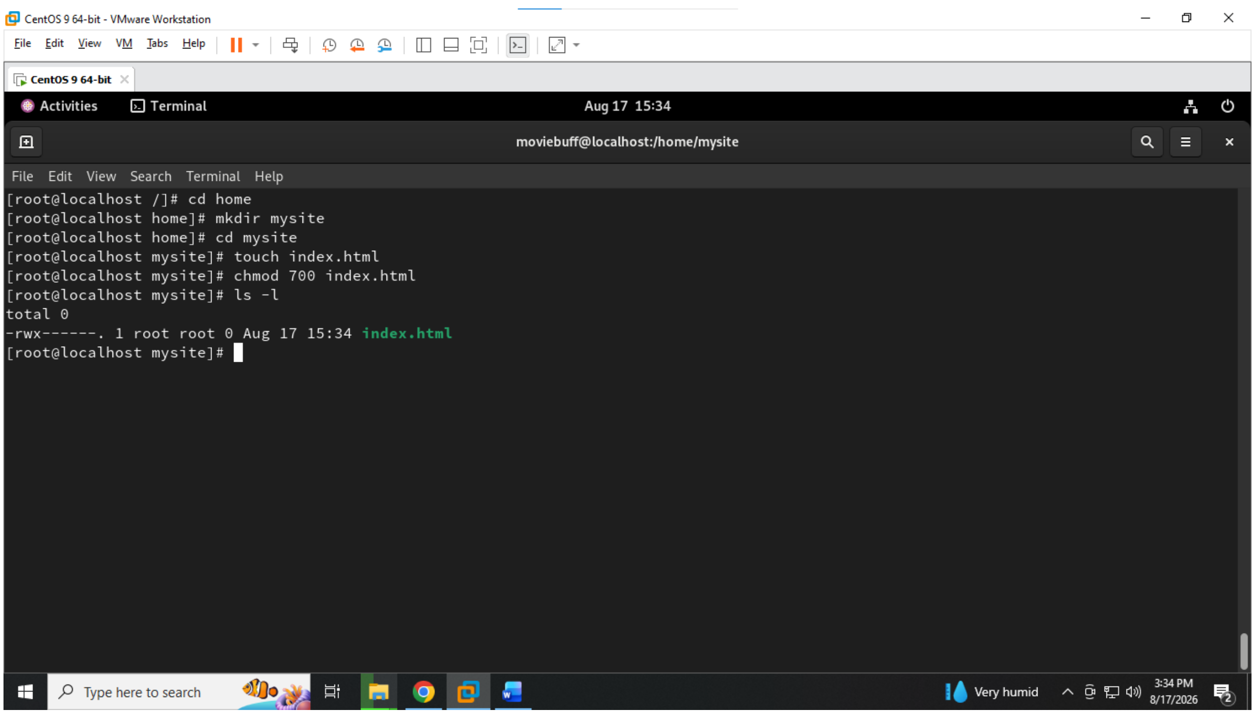
Module-4 : Introduction to Linux Services

Guided by : Vipul mistri

Q.1 Create a new directory called mysite in your home folder, place an index.html file inside it, and set permissions so only your user can read and access the site files.

ANS :

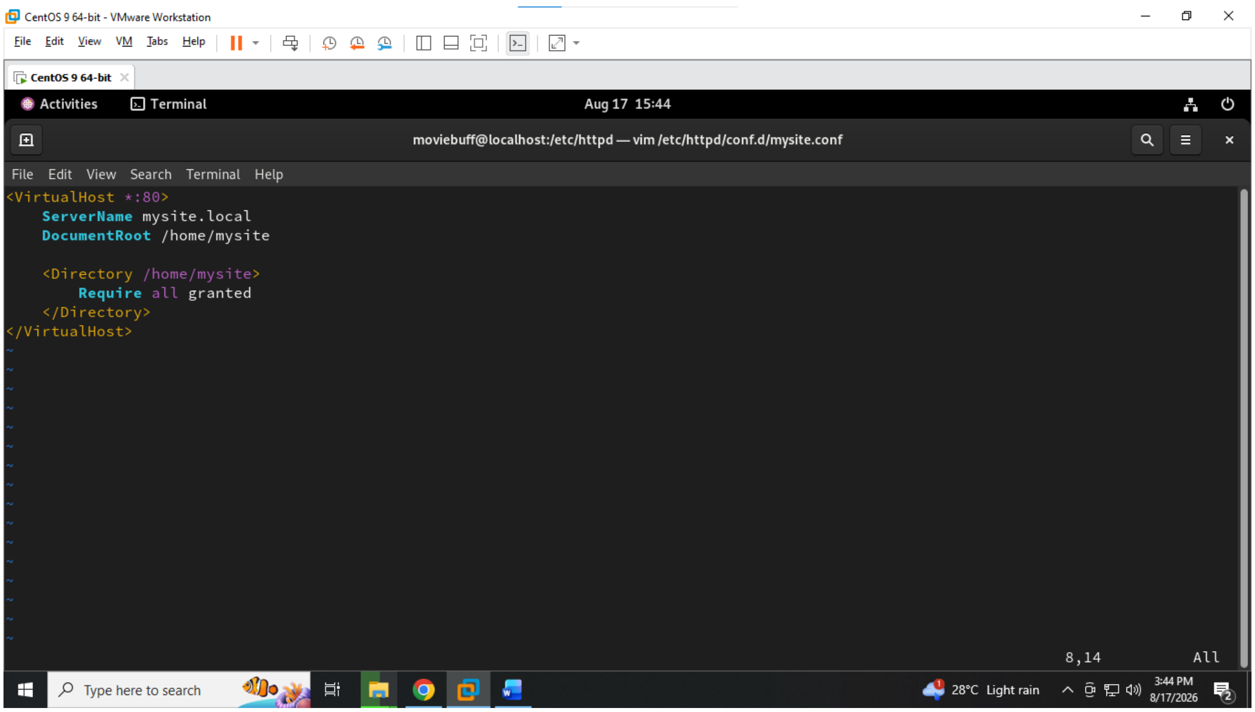


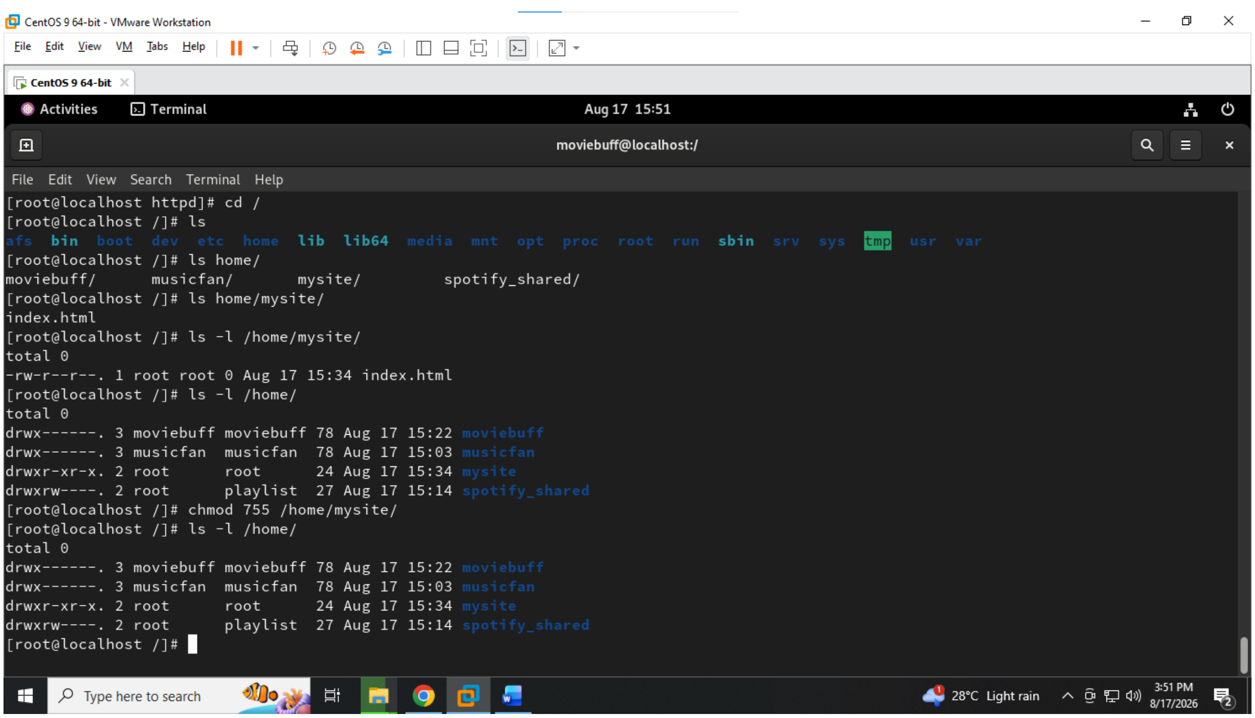
Module-4 : Introduction to Linux Services

Guided by : Vipul mistri

Q.2 Configure your Apache or Nginx server to serve the mysite directory as a website, then test if you can access it from your browser while logged in as your user.

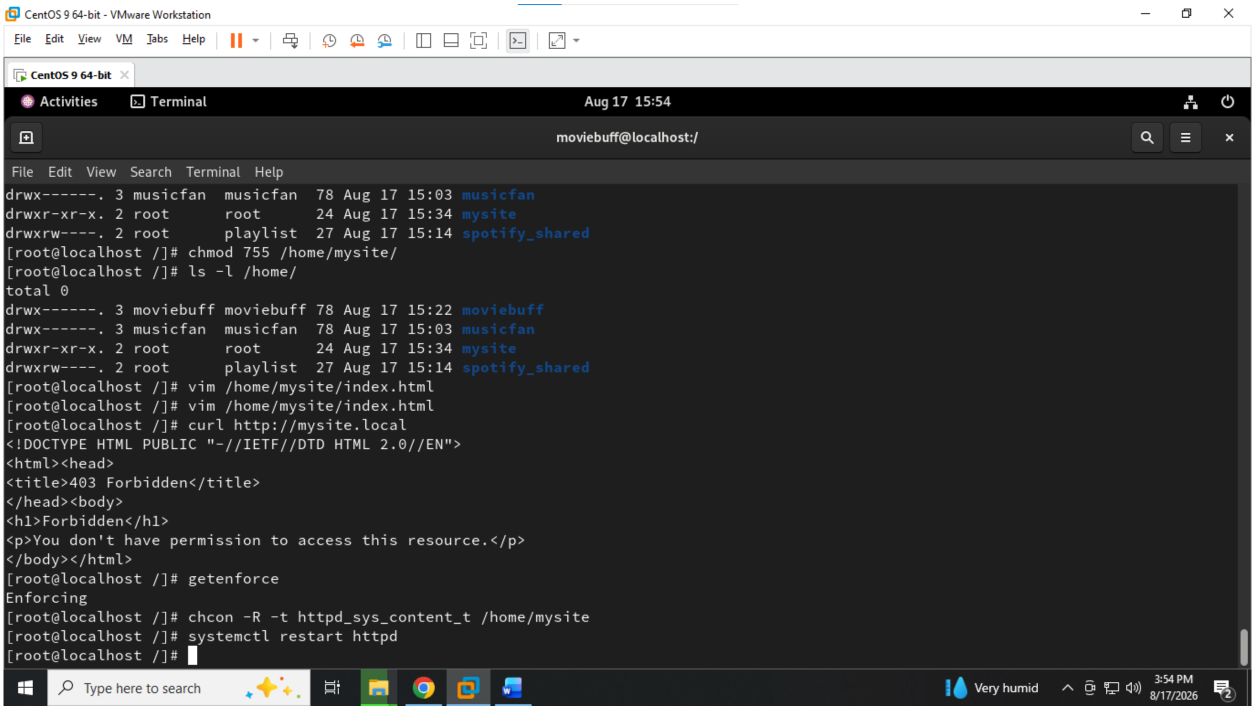
ANS :

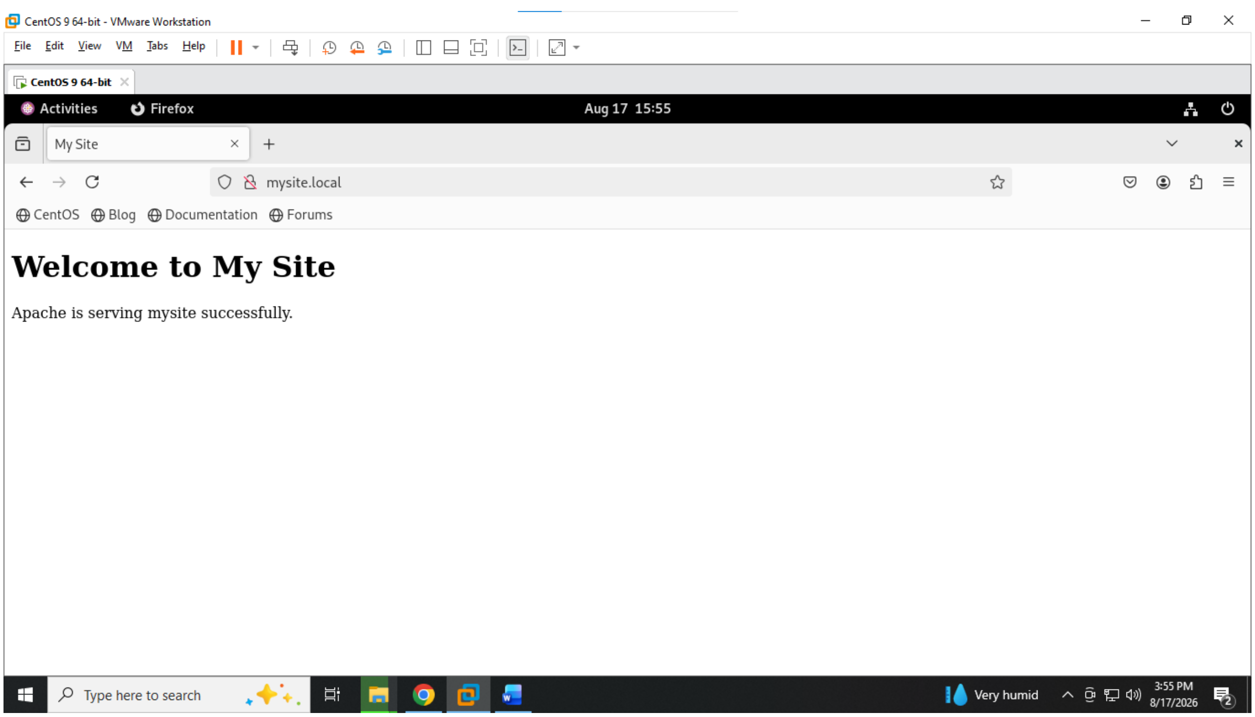




Module-4 : Introduction to Linux Services

Guided by : Vipul mistri



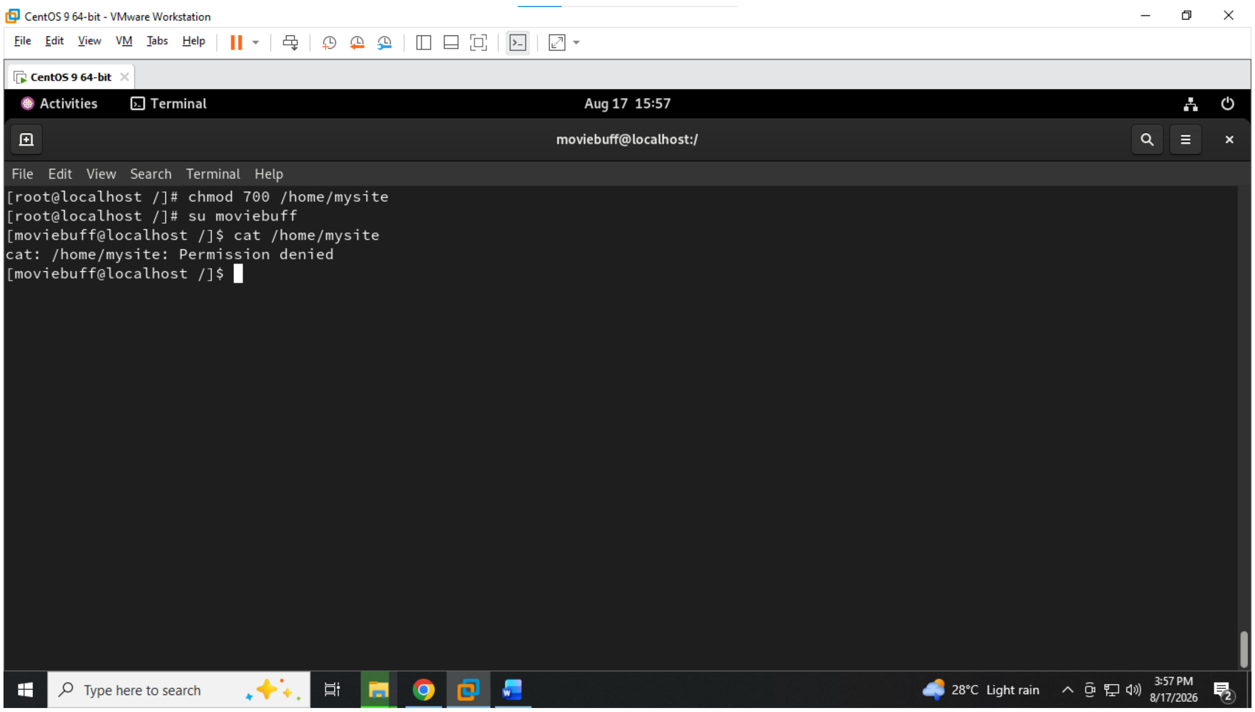


Module-4 : Introduction to Linux Services

Guided by : Vipul mistri

Q.3 Change the permissions of the mysite directory so that another user on the system cannot access the site files directly. Try to access the files as the other user and note the result. Hint: Use chmod and su or a different terminal window for the second user.

ANS :



Permission denied because we have given 700 means only owner have full permission other and groups have no permission to access read write or execute this folder.

Q.4 Document the steps you took to restrict access to your hosted site, including the commands used, permission settings, and screenshots or outputs showing successful and failed access attempts.

ANS : **Restricting Access to the Hosted Apache Website**

Objective

The objective was to configure an Apache-hosted website and control access to the website using Linux file and directory permissions. The website was hosted from /home/mysite.

1. Website Directory

The website was created in:

/home/mysite

The directory contained:

index.html

The Apache configuration was created at:

/etc/httpd/conf.d/mysite.conf

The configuration used:

```
<VirtualHost *:80>
    ServerName mysite.local
    DocumentRoot /home/mysite

    <Directory /home/mysite>
        Require all granted
    </Directory>
</VirtualHost>
```

2. Checking Apache Configuration

The configuration was tested with:

sudo apachectl configtest

The output was:

Syntax OK

Apache was then restarted:

systemctl restart httpd

3. Configuring Hostname Access

The hostname was added to /etc/hosts:

127.0.0.1 mysite.local

This allowed the site to be accessed through:

http://mysite.local

4. Permission Settings

The website directory initially required permission changes so Apache could access the files.

The directory permission was set to:

chmod 755 /home/mysite

The HTML file permission was set to:

chmod 644 /home/mysite/index.html

The 755 permission allows the directory to be traversed and read, while 644 allows the HTML file to be read by Apache without giving other users write permission.

5. Failed Access Attempt

After configuring the hostname, Firefox successfully reached mysite.local, but the browser displayed:

403 Forbidden

You don't have permission to access this resource.

This demonstrated that Apache was running and the hostname was resolving, but access to /home/mysite was still being denied because of the filesystem permissions.

The directory permissions were then corrected using:

chmod 755 /home/mysite

and the HTML file was configured with:

chmod 644 /home/mysite/index.html

6. SELinux Access

Because CentOS uses SELinux, its status was checked with:

getenforce

If SELinux was enforcing, the appropriate Apache content context was applied:

chcon -R -t httpd_sys_content_t /home/mysite

Apache was then restarted:

systemctl restart httpd

7. Access Verification

The website could be tested from the terminal using:

curl -I http://mysite.local

A successful configuration should return:

HTTP/1.1 200 OK

The website can also be verified in Firefox by opening:

http://mysite.local

Conclusion

The Apache website was configured to serve /home/mysite using a VirtualHost. Access was controlled through Linux directory and file permissions. The failed browser attempt produced a **403 Forbidden** error, demonstrating that Apache initially could not access the website content. After correcting the permissions and, where necessary, the SELinux context, Apache was able to access the hosted content. The Apache configuration itself was verified successfully with the **Syntax OK** output.